

JavaScript Mastery Masterclass

From Zero to Advanced Concepts

COMPREHENSIVE TEXTBOOK EDITION

Powered by programenote.dev

Syllabus & Table of Contents

1. What is JavaScript?
2. Hello World: Your First Code
3. Comments in JavaScript
4. Values and Data Types
5. Introduction to Variables
6. Updating Variables (let)
7. Constant Variables (const)
8. The Old Way: var
9. Undefined vs Null
10. Basic Math Operators
11. String Concatenation
12. Assignment Operators
13. Comparison Operators
14. Equality: Loose (==) vs Strict (===)
15. Logical Operators (AND, OR, NOT)
16. Control Flow: The if Statement
17. The if...else Statement
18. Multiple Conditions: else if
19. Truthy and Falsy Values
20. The Switch Statement
21. Loops: Introduction to Iteration
22. The for Loop
23. Break and Continue
24. Introduction to Functions
25. Function Parameters and Arguments

26. Return Values
27. Variable Scope: Global vs Local
28. Block Scope (let & const)
29. Arrays: Introduction
30. Modifying Arrays
31. Array Methods: pop, shift, and unshift
32. Iterating Arrays with For Loops
33. Introduction to Objects
34. Modifying Objects
35. Object Methods
36. Introduction to the DOM
37. Selecting DOM Elements
38. Modifying Text and HTML
39. Modifying CSS Styles via JS
40. Introduction to Events
41. The Event Object
42. Asynchronous JavaScript Introduction
43. The Problem: Callback Hell
44. Introduction to Promises
45. Consuming Promises (.then & .catch)
46. Async and Await (Modern JS)
47. Handling Errors with try/catch
48. The Fetch API

1. What is JavaScript?

JavaScript is a high-level, interpreted, and dynamically typed programming language. Originally created in 1995 by Brendan Eich to add basic interactivity to web pages, it has evolved into a powerful language used on both the client side (browsers) and the server side (Node.js).

Unlike languages like C++ or Java which require you to compile your code before running it, JavaScript is interpreted. This means the browser's JavaScript Engine reads and executes the code line-by-line.

Why Learn JavaScript?

- **Ubiquity:** It is the only programming language native to the web browser.
- **Versatility:** You can build web apps, mobile apps, desktop apps, and servers with it.
- **Community:** It has the largest ecosystem of libraries and frameworks (like React, Angular, Vue).

Live Output:

Terminal / Console

```
> JS Engine is ready.  
> Waiting for instructions...
```

Code Implementation:

```
console.log('JavaScript is running!');
```

2. Hello World: Your First Code

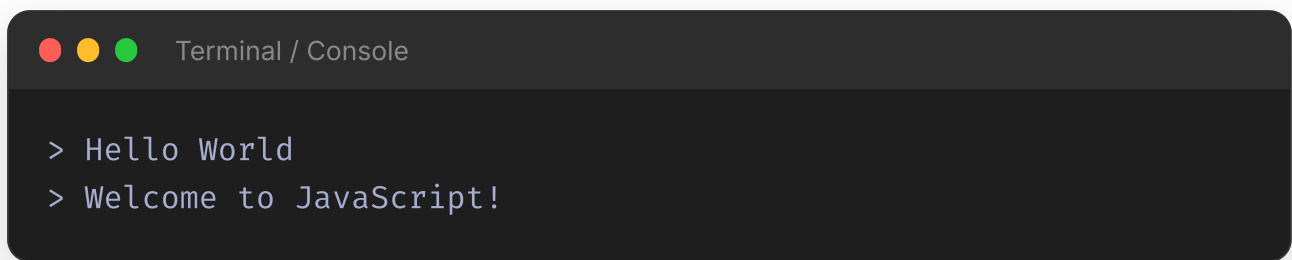
In programming, it is a tradition to write a "Hello World" program as your very first piece of code. In JavaScript, we communicate with the developer console using a built-in object called **console**.

The `console.log()` method is used to print messages to the web console. This is the most essential tool for a developer, as it allows you to see the output of your code, inspect values, and debug errors.

How it works:

When the JavaScript engine encounters `console.log('Hello World');`, it takes the data inside the parentheses and outputs it directly to the digital screen (the console).

Live Output:

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the text "Terminal / Console". The terminal shows two lines of output: "> Hello World" and "> Welcome to JavaScript!".

```
Terminal / Console

> Hello World
> Welcome to JavaScript!
```

Code Implementation:

```
console.log('Hello World');
console.log("Welcome to JavaScript!");
```

3. Comments in JavaScript

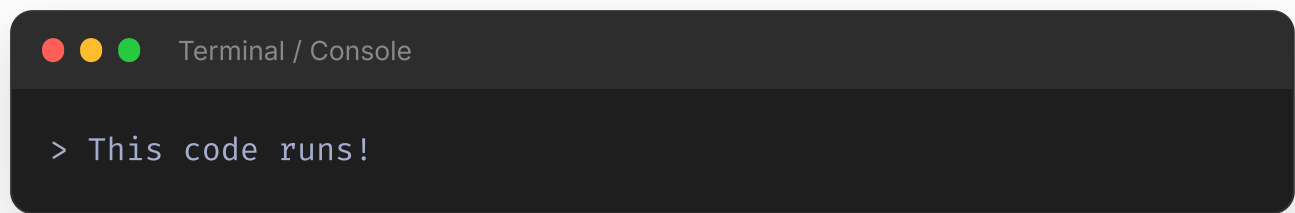
When writing code, you often want to leave notes for yourself or other developers explaining what a piece of code does. These notes are called **Comments**.

Comments are completely ignored by the JavaScript engine when the code runs. They exist purely for human readability.

Two Types of Comments

- **Single-line Comments (`//`)**: Anything written after two forward slashes on that specific line is ignored.
- **Multi-line Comments (`/* ... */`)**: Anything enclosed between these symbols is ignored, allowing you to write paragraph-long explanations.

Live Output:

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the text "Terminal / Console". The terminal shows a single line of output: "> This code runs!".

```
Terminal / Console

> This code runs!
```

Code Implementation:

```
// This is a single-line comment. The engine ignores it.

console.log('This code runs!'); // You can also put comments
at the end of a line.

/*
  This is a multi-line comment.
  It is very useful for explaining complex logic
  or temporarily disabling large blocks of code.
*/
```

4. Values and Data Types

Every piece of information in JavaScript is a **Value**. To make sense of different types of information, JavaScript categorizes values into **Data Types**.

There are two main categories of data types in JavaScript: Primitives and Objects. For now, we will focus on the fundamental Primitives:

- **String:** Used for text. Always enclosed in quotes.
- **Number:** Used for decimals and integers. No quotes.
- **Boolean:** A logical type that can only be `true` or `false`.

By using the `typeof` operator, we can ask JavaScript to tell us the data type of any given value.

Live Output:

Terminal / Console

```
> string  
> number  
> boolean
```

Code Implementation:

```
// Let's print the type of different values.  
  
console.log(typeof 'Hello');    // This is a String  
console.log(typeof 42);         // This is a Number  
console.log(typeof true);       // This is a Boolean
```

5. Introduction to Variables

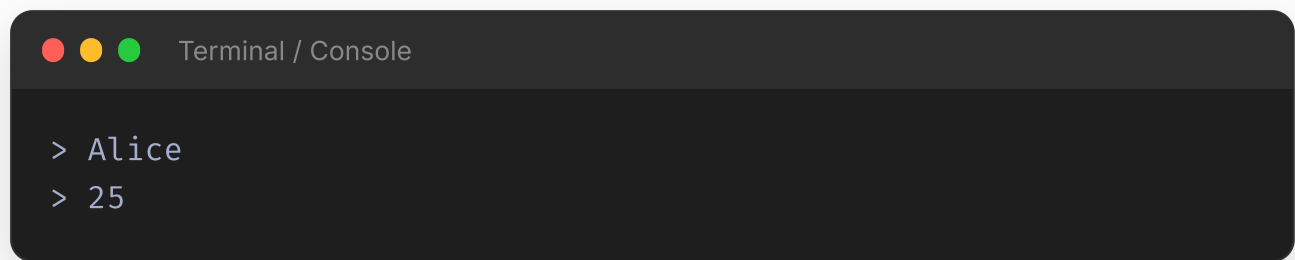
Imagine a variable as a labeled box where you can store a value. Once you store a value in that box, you can reuse it throughout your program by simply referring to the label (the variable name).

In modern JavaScript, we create variables using the `let` keyword. When you declare a variable with `let`, you are telling the computer's memory to reserve a spot for your data.

Rules for Naming Variables:

- Names can contain letters, numbers, underscores, and dollar signs.
- Names **cannot** start with a number.
- Names are case-sensitive (`age` is different from `Age`).
- Use *camelCase* convention (e.g., `firstName`, `totalScore`).

Live Output:

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the text "Terminal / Console". The terminal shows two lines of input: "> Alice" and "> 25".

```
Terminal / Console

> Alice
> 25
```

Code Implementation:

```
// 1. Declare a variable and assign a value to it.
let playerName = 'Alice';
let playerScore = 25;

// 2. Use the variable by referencing its name.
console.log(playerName);
console.log(playerScore);
```

6. Updating Variables (let)

The keyword `let` stands for a variable whose value is allowed to change (mutate) over time. This is incredibly useful for data that constantly updates, like a user's score in a game, or their age.

When updating a variable, you do **not** use the `let` keyword again. You only use `let` when creating the variable for the very first time. To update it, you simply write the variable name followed by the equals sign and the new value.

Behind the scenes, JavaScript finds the 'box' in memory labeled with your variable name, throws out the old value, and puts the new value inside.

Live Output:

Terminal / Console

```
> Starting Score: 0  
> Updated Score: 10  
> Final Score: 15
```

Code Implementation:

```
// Create the variable (Initialization)  
let currentScore = 0;  
console.log('Starting Score:', currentScore);  
  
// Update the variable (Reassignment)  
currentScore = 10;  
console.log('Updated Score:', currentScore);  
  
// Update it again  
currentScore = 15;  
console.log('Final Score:', currentScore);
```

7. Constant Variables (const)

While `let` is great for changing data, sometimes we have data that should **never** change. For example, a person's birth year, or the mathematical value of Pi.

In 2015, JavaScript introduced the `const` keyword. When you declare a variable with `const`, you lock its value. Any attempt to update or reassign a `const` variable will cause your program to crash with an Error.

Best Practice:

Always use `const` by default! Only use `let` if you are absolutely sure that the value will need to change later in the program. This makes your code safer and prevents accidental bugs.

Live Output:

Terminal / Console

```
> 1995  
> Error: Assignment to constant variable.
```

Code Implementation:

```
// Declare a constant variable  
const birthYear = 1995;  
console.log(birthYear);  
  
// If we try to change it, the engine throws an error.  
// Uncommenting the line below will crash the program:  
// birthYear = 1996;
```

8. The Old Way: var

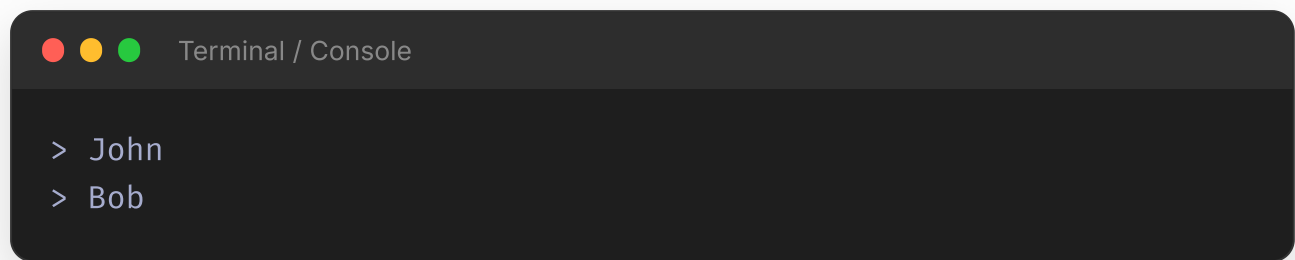
Before 2015, JavaScript only had one way to declare variables: the `var` keyword. You will still see `var` in older codebases and legacy tutorials.

Why did we move away from `var`? Because it has bizarre scoping rules (which we will cover later in the 'Scope' chapter) and allows you to accidentally redeclare the exact same variable twice without throwing an error, which causes massive bugs.

Modern Rule:

Avoid using `var` completely in modern JavaScript. Stick exclusively to `let` and `const`.

Live Output:

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the text "Terminal / Console". The terminal shows two lines of output: "> John" and "> Bob".

```
Terminal / Console

> John
> Bob
```

Code Implementation:

```
// Creating a variable with var
var employeeName = 'John';
console.log(employeeName);

// var allows you to completely redeclare the variable
// without any warnings! (let and const would throw an error
// here)
var employeeName = 'Bob';
console.log(employeeName);
```

9. Undefined vs Null

In JavaScript, there are two special primitives that represent 'nothingness': `undefined` and `null`.

Undefined

When you declare a variable but do not assign a value to it, JavaScript automatically assigns it the value of `undefined`. It means 'this variable exists, but it has no value yet'.

Null

`null` is used when you, the developer, intentionally want to say 'this variable holds absolutely nothing'. It is an intentional absence of value.

Think of it this way: `undefined` is an empty box that you forgot to pack. `null` is an empty box that you deliberately left empty.

Live Output:

Terminal / Console

```
> undefined  
> null
```

Code Implementation:

```
// 1. Undefined (Automatic)  
let futureData;  
console.log(futureData); // We didn't assign a value!  
  
// 2. Null (Manual)  
let emptyResult = null;  
console.log(emptyResult); // We intentionally set it to  
nothing.
```

10. Basic Math Operators

JavaScript can act as a powerful calculator. It includes standard Arithmetic Operators that allow us to perform mathematical calculations on Numbers.

- **Addition (`+`)**: Adds two numbers.
- **Subtraction (`-`)**: Subtracts the right number from the left.
- **Multiplication (`*`)**: Multiplies two numbers.
- **Division (`/`)**: Divides the left number by the right.
- **Exponentiation (`**`)**: Raises the left number to the power of the right number.

Just like in standard mathematics, JavaScript follows the order of operations (BODMAS/PEMDAS). You can use parentheses `()` to control which calculations happen first.

Live Output:

Terminal / Console

```
> Sum: 15  
> Product: 50  
> Power: 8  
> Complex: 14
```

Code Implementation:

```
let a = 10;  
let b = 5;  
  
console.log('Sum:', a + b);  
console.log('Product:', a * b);  
console.log('Power:', 2 ** 3); // 2 to the power of 3  
  
// Using parentheses to control the order of operations  
let result = (10 - 3) * 2;  
console.log('Complex:', result);
```

11. String Concatenation

Strings are text values. When you use the addition operator (`+`) on strings, JavaScript does not perform mathematical addition. Instead, it performs **Concatenation**, which means joining the strings end-to-end.

For example, `'Hello' + 'World'` becomes `'HelloWorld'`.

The Quirk of JavaScript Coercion:

If you try to add a String and a Number together (e.g., `'Score: ' + 10`), JavaScript will secretly convert the Number into a String behind the scenes and concatenate them. This is known as **Implicit Coercion**.

Live Output:

Terminal / Console

```
> Hello John  
> Your score is 95
```

Code Implementation:

```
let greeting = 'Hello';  
let personName = 'John';  
  
// We add a space string ' ' in the middle so they don't  
stick together  
let fullMessage = greeting + ' ' + personName;  
console.log(fullMessage);  
  
let scoreText = 'Your score is ';  
let points = 95;  
  
// Number 95 is coerced into the string '95'  
console.log(scoreText + points);
```

12. Assignment Operators

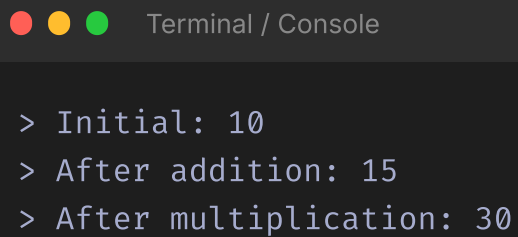
You already know the basic assignment operator (`=`), which assigns the value on the right to the variable on the left.

JavaScript also provides shorthand **Compound Assignment Operators** to make updating variables faster. Instead of writing `score = score + 5`, you can simply write `score += 5`.

- `+=` : Adds and assigns.
- `-=` : Subtracts and assigns.
- `*=` : Multiplies and assigns.
- `/=` : Divides and assigns.

These are heavily used in loops and when maintaining state (like a running total or score).

Live Output:



```
> Initial: 10
> After addition: 15
> After multiplication: 30
```

Code Implementation:

```
let myNumber = 10;
console.log('Initial:', myNumber);

// Long way: myNumber = myNumber + 5;
// Shorthand:
myNumber += 5;
console.log('After addition:', myNumber);

// Multiply the current value by 2
myNumber *= 2;
console.log('After multiplication:', myNumber);
```

13. Comparison Operators

Comparison operators are used to compare two values. They always evaluate to a Boolean result: either `true` or `false`.

- `>` : Greater than.
- `<` : Less than.
- `>=` : Greater than or equal to.
- `<=` : Less than or equal to.

These operators are the foundation of decision-making in programming. Later, we will use them inside Control Flow statements to decide which blocks of code the computer should execute.

Live Output:



```
> Is 10 greater than 5? true
> Is 10 less than 10? false
> Is 10 less than or equal to 10? true
```

Code Implementation:

```
let a = 10;
let b = 5;

console.log('Is 10 greater than 5?', a > b);
console.log('Is 10 less than 10?', a < 10);
console.log('Is 10 less than or equal to 10?', a ≤ 10);
```

14. Equality: Loose (==) vs Strict (===)

Checking if two values are equal is done using equality operators, but JavaScript has two different kinds: Loose Equality and Strict Equality.

Loose Equality (`=`)

Checks if the values are equal, but **ignores the data type**. If the types are different, JavaScript attempts to convert them (coercion) before comparing. `5 = '5'` is `true`.

Strict Equality (`===`)

Checks if BOTH the value and the data type are identical. No conversion happens. `5 === '5'` is `false` because one is a Number and the other is a String.

Golden Rule: Always use `===` (Strict) in modern JavaScript to avoid coercion bugs!

Live Output:

```
Terminal / Console

> Loose check (5 = '5'): true
> Strict check (5 === '5'): false
> Strict check (5 === 5): true
```

Code Implementation:

```
let num = 5;          // Number
let str = '5';        // String

// Loose equality (Coercion happens)
console.log("Loose check (5 = '5'):", num == str);

// Strict equality (No coercion - Safer!)
console.log("Strict check (5 === '5'):", num === str);

// Strict equality with matching types
console.log("Strict check (5 === 5):", num === 5);
```

15. Logical Operators (AND, OR, NOT)

Logical Operators allow you to combine multiple boolean conditions together.

- **AND (&&)**: Returns `true` ONLY if **all** conditions are true. If even one is false, the whole expression becomes false.
- **OR (||)**: Returns `true` if **at least one** condition is true. It only returns false if all conditions are false.
- **NOT (!)**: Reverses the boolean value. `true` becomes `false`, and vice versa.

These are crucial when making complex decisions, like verifying if a user is both an Admin AND has an Active account.

Live Output:

```
Terminal / Console

> Can drive? false
> Needs ticket? true
> Inverted true is: false
```

Code Implementation:

```
let hasLicense = true;
let hasGoodVision = false;

// AND: Both must be true
let canDrive = hasLicense && hasGoodVision;
console.log('Can drive?', canDrive);

// OR: One must be true
let isStudent = true;
let isSenior = false;
let needsDiscountTicket = isStudent || isSenior;
console.log('Needs ticket?', needsDiscountTicket);

// NOT: Inverts the value
console.log('Inverted true is:', !true);
```

16. Control Flow: The if Statement

By default, code executes line-by-line from top to bottom. **Control Flow** allows us to change this path based on conditions. The most fundamental tool for this is the `if` statement.

The `if` statement evaluates a condition inside parentheses `()`. If that condition results in `true`, the code inside the curly braces `{}` (the block) will be executed. If it evaluates to `false`, the block is completely skipped.

This allows your program to be dynamic and react differently to different data (like showing a 'Welcome' message only if the user is logged in).

Live Output:

Terminal / Console

```
> You are an adult.
```

Code Implementation:

```
let userAge = 20;

// The code inside the { } only runs because 20 ≥ 18 is true.
if (userAge ≥ 18) {
  console.log('You are an adult.');
```

```
}

let batteryLevel = 10;

// This block will be skipped because 10 < 5 is false.
if (batteryLevel < 5) {
  console.log('Battery critical! Please charge.');
```

```
}
```

17. The if...else Statement

While the `if` statement handles what to do when a condition is true, the `else` statement acts as the fallback. It handles what to do when the condition is **false**.

An `else` block cannot exist on its own; it must immediately follow an `if` block. This structure guarantees that exactly **one** of the two blocks will execute, making it a perfect tool for binary decisions (e.g., Login Success vs Login Failed).

Live Output:

 Terminal / Console

```
> Access Denied. Please check your password.
```

Code Implementation:

```
let isPasswordCorrect = false;

if (isPasswordCorrect === true) {
  // Runs if the condition is true
  console.log('Login Successful! Welcome.');
```

```
} else {
  // Runs if the condition is false
  console.log('Access Denied. Please check your password.');
```

```
}
```

18. Multiple Conditions: else if

Sometimes a decision isn't just binary (True/False). You might have multiple distinct scenarios. This is where `else if` comes in.

You can chain as many `else if` blocks as you want between the initial `if` and the final fallback `else`. JavaScript will evaluate them from top to bottom. As soon as it finds a **true** condition, it executes that block and immediately **exits the entire chain**, skipping the rest.

This is extremely useful for grading systems, categorizing data, or handling multiple specific error codes.

Live Output:

Terminal / Console

```
> Letter Grade: B
```

Code Implementation:

```
let testScore = 85;

if (testScore ≥ 90) {
  console.log('Letter Grade: A');
} else if (testScore ≥ 80) {
  // This block runs! 85 is ≥ 80.
  console.log('Letter Grade: B');
} else if (testScore ≥ 70) {
  // This is skipped because a match was already found
  above.
  console.log('Letter Grade: C');
} else {
  console.log('Letter Grade: F');
}
```

19. Truthy and Falsy Values

In JavaScript, `if` conditions don't strictly require a boolean `true` or `false`. You can put **any** data type inside an `if` statement, and JavaScript will automatically coerce it into a boolean.

Falsy Values:

There are exactly 6 values that evaluate to `false`: `0`, `""` (empty string), `null`, `undefined`, `NaN` (Not a Number), and of course, `false`.

Truthy Values:

Literally **everything else** evaluates to `true`. This includes numbers like `1` or `-5`, any non-empty string like `'hello'`, and even empty arrays `[]` or objects `{}`.

Live Output:

Terminal / Console

```
> This string is Truthy!  
> 0 is Falsy, so the else block runs.
```

Code Implementation:

```
let myName = 'Nitesh';  
  
// 'Nitesh' is a non-empty string, so it is Truthy.  
if (myName) {  
  console.log('This string is Truthy!');  
}  
  
let itemsInCart = 0;  
  
// 0 is Falsy, so the if block is skipped.  
if (itemsInCart) {  
  console.log('You have items to checkout.');
```

```
} else {  
  console.log('0 is Falsy, so the else block runs.');
```

```
}
```

20. The Switch Statement

When you have a long chain of `else if` statements testing the same variable against different strict values, it can look messy. The `switch` statement provides a cleaner syntax for this.

The `switch` takes a single value and evaluates it against multiple `case` blocks using strict equality (`===`). When a match is found, it executes the code.

The Break Keyword:

Crucially, you must use the `break` keyword at the end of each case. If you forget `break`, JavaScript will "fall through" and execute all the code in the subsequent cases as well, regardless of whether they match!

Live Output:

Terminal / Console

```
> The traffic light is Yellow: Slow down!
```

Code Implementation:

```
let trafficLight = 'Yellow';

switch (trafficLight) {
  case 'Red':
    console.log('Stop!');
    break;
  case 'Yellow':
    console.log('The traffic light is Yellow: Slow down!');
    break; // Stops execution here
  case 'Green':
    console.log('Go!');
    break;
  default:
    // Runs if no cases match
    console.log('Invalid color.');
}
```

21. Loops: Introduction to Iteration

In programming, we often need to perform the exact same task multiple times. Instead of copy-pasting the same line of code over and over (which violates the DRY principle: Don't Repeat Yourself), we use **Loops**.

Loops allow us to execute a block of code repeatedly as long as a certain condition remains true. The moment the condition becomes false, the loop stops, and the program continues.

The Danger of Infinite Loops

If you create a loop where the condition *never* becomes false, the loop will run forever. This is called an Infinite Loop, and it will crash the browser or the application by consuming all available memory and CPU power.

Live Output:

Terminal / Console

```
> Iteration 0  
> Iteration 1  
> Iteration 2  
> Loop finished!
```

Code Implementation:

```
let count = 0;  
  
// A basic loop structure (using while)  
while (count < 3) {  
  console.log('Iteration', count);  
  // Crucial step: we must change the variable  
  // so the condition eventually becomes false!  
  count = count + 1;  
}  
  
console.log('Loop finished!');
```

22. The for Loop

The `for` loop is the most commonly used loop in JavaScript. It is perfect for situations where you know exactly how many times you want the code to repeat (for example, counting from 1 to 10).

The `for` loop consolidates three crucial pieces of looping logic into a single line:

1. **Initialization:** Creating the counter variable (e.g., `let i = 0;`). This runs only once at the start.
2. **Condition:** The rule that keeps the loop running (e.g., `i < 5;`). Checked before every iteration.
3. **Increment/Decrement:** Updating the counter (e.g., `i++`). Runs at the end of every iteration.

Live Output:

Terminal / Console

```
> 1  
> 2  
> 3  
> Blastoff!
```

Code Implementation:

```
// (Initialization; Condition; Increment)  
for (let i = 1; i ≤ 3; i++) {  
  // This code runs as long as i ≤ 3  
  console.log(i);  
}  
  
// The loop finishes when i becomes 4 (4 ≤ 3 is false)  
console.log('Blastoff!');
```

23. Break and Continue

Loops don't always have to run to their natural end. JavaScript provides two powerful keywords to manually control the flow inside a loop: `break` and `continue`.

Break

The `break` keyword immediately **kills** the loop entirely. It is often used when searching for a specific item. Once the item is found, there's no need to keep looping, so you `break` out to save processing power.

Continue

The `continue` keyword skips the **current iteration** and immediately jumps back to the top of the loop to evaluate the condition for the next cycle. It is used to skip over data you don't care about without killing the whole loop.

Live Output:

```
Terminal / Console

> 1
> 2
> Found it! Stopping loop.
> ≡≡≡ Next Example ≡≡≡
> 1
> 3
> 5
```

Code Implementation:

```
for (let i = 1; i ≤ 5; i++) {
  if (i ≡≡≡ 3) {
    console.log('Found it! Stopping loop.');
```

```
    break; // Kills the loop completely
  }
  console.log(i);
}

console.log('≡≡≡ Next Example ≡≡≡');
```

```
for (let n = 1; n ≤ 5; n++) {
  if (n % 2 ≡≡≡ 0) {
    // If the number is even, skip to the next iteration
    continue;
  }
  console.log(n); // Only prints odd numbers
}
```

24. Introduction to Functions

A Function is a reusable block of code designed to perform a specific task. Think of it as a mini-program within your main program.

Why use Functions?

If you have a block of logic that you need to use in 5 different places, you shouldn't copy-paste it 5 times. Instead, you wrap it inside a Function. Then, you simply "call" or "invoke" that function whenever you need it.

Anatomy of a Function:

- **Declaration:** Use the `function` keyword, give it a name, and add `()` and `{}`.
- **Invocation:** To actually run the code inside, you must call the function by writing its name followed by parentheses: `myFunction()`.

Live Output:

Terminal / Console

```
> Initializing System...  
> Welcome to the Matrix.  
> System Ready.
```

Code Implementation:

```
// 1. Declaring the function (This does NOT run the code yet)  
function bootSystem() {  
    console.log('Initializing System... ');  
    console.log('Welcome to the Matrix.');
```

```
    console.log('System Ready.');
```

```
}  
  
// 2. Invoking the function (This runs the code)  
bootSystem();
```

25. Function Parameters and Arguments

Functions become truly powerful when they can accept data from the outside, process it, and act dynamically based on that data.

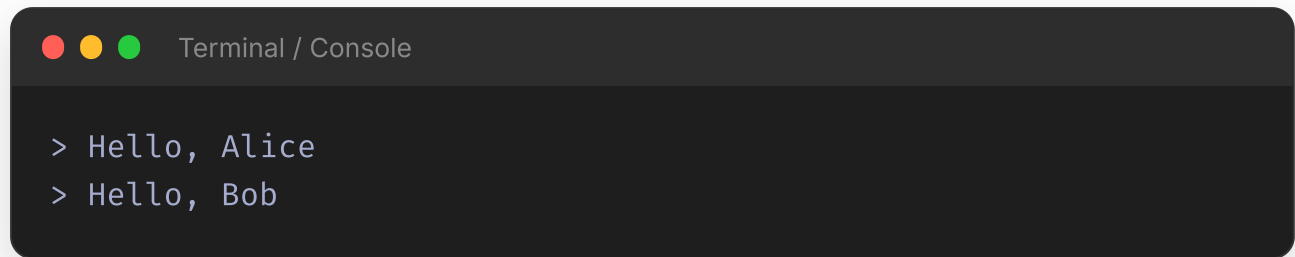
Parameters (The placeholders)

When you declare a function, you can define variables inside the parentheses. These are called **Parameters**. They act as placeholders for the data the function expects to receive.

Arguments (The actual data)

When you invoke the function, the actual values you pass into the parentheses are called **Arguments**. JavaScript maps the arguments to the parameters in the exact order they are passed.

Live Output:

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the text "Terminal / Console". The terminal shows two lines of output: "> Hello, Alice" and "> Hello, Bob".

```
Terminal / Console

> Hello, Alice
> Hello, Bob
```

Code Implementation:

```
// 'userName' is a Parameter (a placeholder variable)
function greetUser(userName) {
  console.log('Hello, ' + userName);
}

// 'Alice' is the Argument (the actual data)
greetUser('Alice');

// We can reuse the function with different data!
greetUser('Bob');
```

26. Return Values

So far, our functions have only printed things to the console. But in real-world applications, functions usually calculate a result and send it back to the main program to be used later. We do this using the `return` keyword.

When JavaScript encounters a `return` statement, two things happen immediately:

1. The function completely stops executing (any code below the return is ignored).
2. The value specified is "spit out" (returned) to wherever the function was called.

You can then store this returned value in a variable to use it elsewhere.

Live Output:

Terminal / Console

```
> Result of 5 + 10 is: 15  
> Result of 100 + 200 is: 300
```

Code Implementation:

```
function addNumbers(a, b) {  
  let sum = a + b;  
  // Send the result back to the caller  
  return sum;  
  
  // This will never run!  
  // console.log('I am invisible');  
}  
  
// Store the returned value in a variable  
let finalResult = addNumbers(5, 10);  
console.log('Result of 5 + 10 is:', finalResult);  
  
let anotherResult = addNumbers(100, 200);  
console.log('Result of 100 + 200 is:', anotherResult);
```

27. Variable Scope: Global vs Local

Scope defines the accessibility (visibility) of variables. Not all variables are accessible from everywhere in your code.

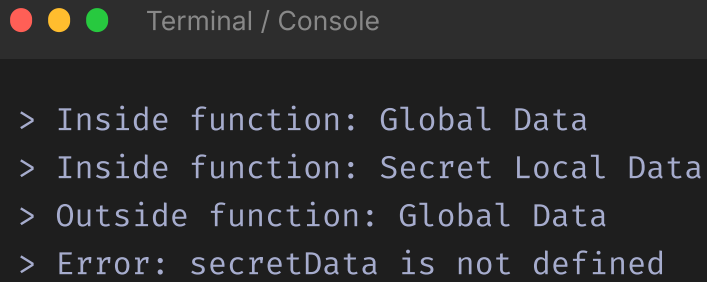
Global Scope

A variable declared outside of any function has Global Scope. It can be accessed and modified from *anywhere* in your JavaScript file. This can be dangerous if overused.

Local (Function) Scope

Variables declared *inside* a function are Local to that function. They are completely hidden from the outside world. They are created when the function starts and destroyed when the function ends. This provides security and memory efficiency.

Live Output:



```
> Inside function: Global Data
> Inside function: Secret Local Data
> Outside function: Global Data
> Error: secretData is not defined
```

Code Implementation:

```
let globalData = 'Global Data';

function checkScope() {
  // We can access global variables inside functions
  console.log('Inside function:', globalData);

  let secretData = 'Secret Local Data';
  console.log('Inside function:', secretData);
}

checkScope();

// We can access global data outside
console.log('Outside function:', globalData);

// But we CANNOT access local data outside!
// Uncommenting the below line crashes the app:
// console.log(secretData);
```

28. Block Scope (let & const)

Before ES6 (2015), JavaScript only had Global and Function scope. This meant that if you created a variable inside an `if` block or a `for` loop using `var`, it would "leak out" and be accessible outside the block. This was a massive source of bugs.

With the introduction of `let` and `const`, JavaScript finally got **Block Scope**.

A Block is any code bounded by curly braces `{}`. Any variable declared with `let` or `const` inside a block is strictly confined to that block. Once the block finishes executing, the variable is destroyed.

Live Output:

Terminal / Console

```
> Inside block: Password123  
> Error: userPassword is not defined
```

Code Implementation:

```
let isLoggedIn = true;  
  
if (isLoggedIn === true) {  
  // This variable is block-scoped!  
  let userPassword = 'Password123';  
  console.log('Inside block:', userPassword);  
}  
  
// We cannot access 'userPassword' here because the block  
// ended.  
// The variable 'leaking' is prevented by using 'let'.  
// Uncommenting below will throw a ReferenceError:  
// console.log(userPassword);
```

29. Arrays: Introduction

Variables are great for storing single values, but what if you need to store a list of 100 students? Creating 100 variables (student1, student2, etc.) is impossible to manage.

An **Array** is a special data structure that allows you to store a collection of multiple values in a single variable. Think of it as a bookshelf where each book has a specific position.

Zero-Indexed

The most important thing to remember about arrays in programming is that they are **Zero-Indexed**. The very first item in the list is at position 0, the second item is at position 1, and so on.

Live Output:

Terminal / Console

```
> Apple  
> Banana  
> Total fruits: 3
```

Code Implementation:

```
// Create an Array using square brackets [ ]  
let fruits = ['Apple', 'Banana', 'Mango'];  
  
// Access items using their Index number in square brackets  
console.log(fruits[0]); // Gets the 1st item (Apple)  
console.log(fruits[1]); // Gets the 2nd item (Banana)  
  
// Arrays have a built-in property called .length  
// It tells you exactly how many items are in the list.  
console.log('Total fruits:', fruits.length);
```

30. Modifying Arrays

Arrays are mutable, meaning you can change their contents after they are created. You can update existing elements by targeting their index, or you can add completely new elements.

Updating

Target the specific index (e.g., `arr[1]`) and assign a new value to it using the `=` operator. The old value is overwritten.

Adding with Built-in Methods

Instead of manually finding the last index to add an item, JavaScript provides built-in functions (methods) attached to every array. The most common is `push()`, which seamlessly adds a new item to the very end of the array.

Live Output:

Terminal / Console

```
> Original: ['Red', 'Blue']  
> Updated: ['Red', 'Green']  
> After Push: ['Red', 'Green', 'Yellow']
```

Code Implementation:

```
let colors = ['Red', 'Blue'];  
console.log('Original:', colors);  
  
// 1. Update an existing item (replace Blue with Green)  
colors[1] = 'Green';  
console.log('Updated:', colors);  
  
// 2. Add a new item to the end using the .push() method  
colors.push('Yellow');  
console.log('After Push:', colors);
```

31. Array Methods: pop, shift, and unshift

Beyond `push()` which adds to the end, JavaScript provides three more essential methods for manipulating the elements of an array at its boundaries.

pop()

Removes the **last** element from an array. It also returns that removed element in case you want to save it in a variable.

shift()

Removes the **first** element from an array (at index 0). All other elements are shifted down by one index.

unshift()

Adds one or more elements to the **beginning** of an array. All other elements are shifted up.

Live Output:

Terminal / Console

```
> Original: [10, 20, 30]
> After pop: [10, 20]
> After unshift: [5, 10, 20]
> After shift: [10, 20]
```

Code Implementation:

```
let numbers = [10, 20, 30];
console.log('Original:', numbers);

// Remove the last item (30)
numbers.pop();
console.log('After pop:', numbers);

// Add item to the start (5)
numbers.unshift(5);
console.log('After unshift:', numbers);

// Remove item from the start (5)
numbers.shift();
console.log('After shift:', numbers);
```

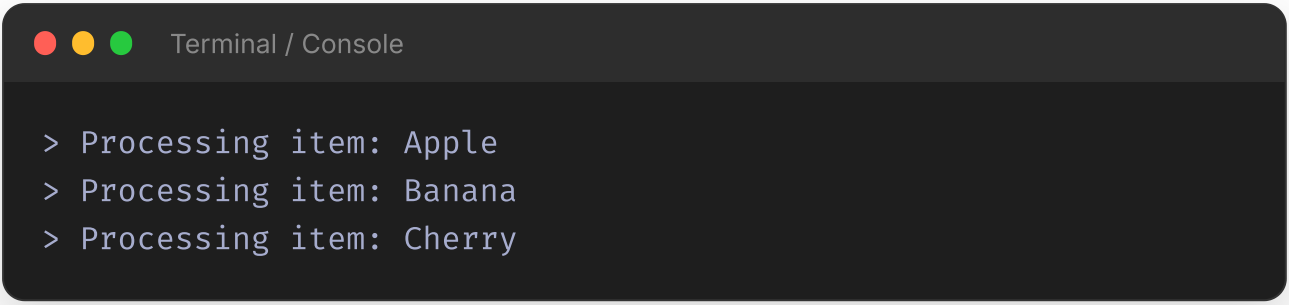
32. Iterating Arrays with For Loops

Arrays and Loops are best friends in programming. An array holds a list of data, and a loop allows you to automatically execute code for every single piece of data in that list.

We can use a standard `for` loop to iterate over an array. The trick is to start the loop counter `i` at 0 (since arrays are zero-indexed), and set the condition to run as long as `i` is less than the array's `length`.

Inside the loop, we use `array[i]` to dynamically access the current element as the loop progresses.

Live Output:



```
Terminal / Console  
  
> Processing item: Apple  
> Processing item: Banana  
> Processing item: Cherry
```

Code Implementation:

```
let cart = ['Apple', 'Banana', 'Cherry'];  
  
// i starts at 0.  
// It runs as long as i < 3 (the length of the array)  
// i increments by 1 each time.  
for (let i = 0; i < cart.length; i++) {  
  
    // cart[i] will be cart[0], then cart[1], then cart[2]  
    let currentItem = cart[i];  
  
    console.log('Processing item:', currentItem);  
}
```

33. Introduction to Objects

Arrays are great for ordered lists, but they are terrible if you want to store complex, structured data where each piece of information has a specific label. For this, we use **Objects**.

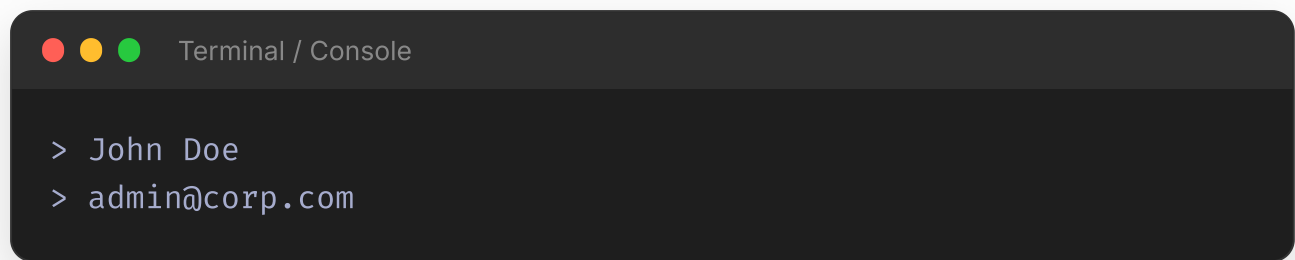
An Object is a collection of Key-Value pairs. It is declared using curly braces `{ }`.

Keys and Values

The **Key** (also called a Property name) acts as a label. The **Value** is the data associated with that label. Values can be anything: Strings, Numbers, Arrays, or even other Objects!

Objects are the most important data structure in JavaScript. They allow us to group related data together logically (e.g., storing a User's name, email, and age in one place).

Live Output:

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the text "Terminal / Console". The terminal shows two lines of input: "> John Doe" and "> admin@corp.com".

```
Terminal / Console

> John Doe
> admin@corp.com
```

Code Implementation:

```
// Creating an object using curly braces
let userProfile = {
  // Key: Value
  name: 'John Doe',
  email: 'admin@corp.com',
  age: 30,
  isActive: true
};

// Accessing object data using Dot Notation (.)
console.log(userProfile.name);
console.log(userProfile.email);
```

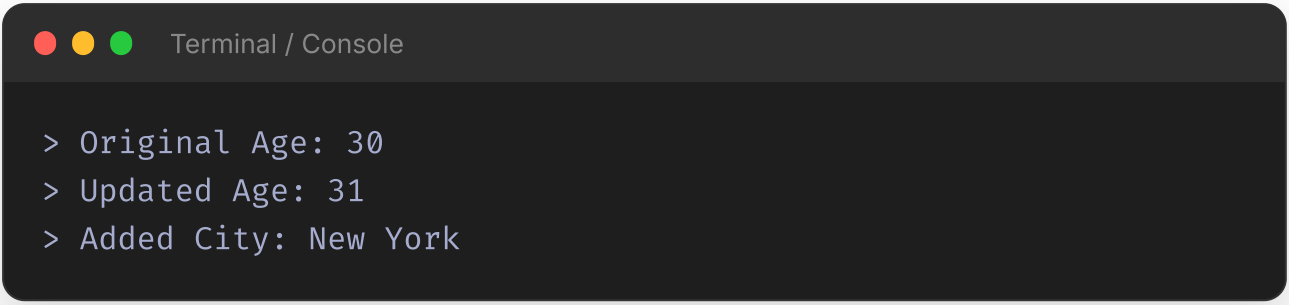
34. Modifying Objects

Just like Arrays, Objects are mutable. You can easily add new properties, update existing properties, or even delete properties entirely after the object has been created.

Dot Notation vs Bracket Notation

You can modify objects using standard Dot Notation (e.g., `user.age = 31`). However, if your Key has a space in it, or if you are using a variable to dynamically select a Key, you must use **Bracket Notation** (e.g., `user['age'] = 31`).

Live Output:

A terminal window with a dark background and a title bar containing three colored circles (red, yellow, green) and the text "Terminal / Console". The terminal displays three lines of output: "> Original Age: 30", "> Updated Age: 31", and "> Added City: New York".

```
Terminal / Console

> Original Age: 30
> Updated Age: 31
> Added City: New York
```

Code Implementation:

```
let user = {
  name: 'John',
  age: 30
};
console.log('Original Age:', user.age);

// 1. Updating an existing property
user.age = 31;
console.log('Updated Age:', user.age);

// 2. Adding a completely new property
user.city = 'New York';
console.log('Added City:', user.city);

// 3. Using bracket notation for dynamic keys
let keyToUpdate = 'age';
user[keyToUpdate] = 32;
```

35. Object Methods

Objects don't just store static data (like strings and numbers). They can also store Functions! When a function is stored inside an object as a property, we call it a **Method**.

Methods allow objects to have *behaviors*. For example, a 'User' object might have a 'login' method, or a 'Car' object might have a 'drive' method.

The 'this' Keyword

Inside an object method, the keyword `this` acts as a reference to the object itself. It allows the method to access other properties belonging to the same object (e.g., `this.name`).

Live Output:

Terminal / Console

```
> Engine started. Vroom!  
> The Toyota is driving.
```

Code Implementation:

```
let car = {  
  brand: 'Toyota',  
  color: 'Red',  
  
  // This is a Method  
  start: function() {  
    console.log('Engine started. Vroom!');  
  },  
  
  // Method using 'this' to access object properties  
  drive: function() {  
    console.log('The ' + this.brand + ' is driving.');  }  
};  
  
// Calling the methods  
car.start();  
car.drive();
```

36. Introduction to the DOM

JavaScript was created to make web pages interactive. But how does JavaScript actually talk to the HTML? Through the **DOM** (Document Object Model).

When a web browser loads an HTML file, it translates all the HTML tags (like `<h1>` , `<div>` , `<p>`) into a massive JavaScript Object. This structured object tree is the DOM.

The Document Object

The entry point to the DOM is a globally available object called `document` . By using the `document` object in JavaScript, we can read the page structure, change text, change colors, or add completely new elements dynamically!

Live Output:

Terminal / Console

```
> [Digital Console Representation]
> Accessing the DOM...
> document.title holds the page title.
```

Code Implementation:

```
// The document object gives us access to the entire HTML
page.

// We can read properties of the page
let pageTitle = document.title;
console.log('The title of this page is:', pageTitle);

// We can read the URL
let currentURL = document.URL;
console.log('URL:', currentURL);
```

37. Selecting DOM Elements

Before we can change a heading or a button on our webpage, we must first tell JavaScript which element we want to modify. We do this by "selecting" elements from the DOM.

Methods for Selecting:

- `getElementById()` : Finds a single element by its unique HTML `id` attribute.
- `querySelector()` : The most modern and powerful method. It uses CSS selectors (like `.class` or `#id`) to find the first matching element.
- `querySelectorAll()` : Finds ALL elements matching a CSS selector and returns them in a list.

Live Output:

Terminal / Console

```
> Selected Heading Element  
> Selected Button Element
```

Code Implementation:

```
// Assuming our HTML has: <h1 id="main-title">Hello</h1>  
// and: <button class="submit-btn">Submit</button>  
  
// 1. Select by ID  
let titleElement = document.getElementById('main-title');  
console.log('Selected Heading Element');  
  
// 2. Select using a CSS class selector  
let buttonElement = document.querySelector('.submit-btn');  
console.log('Selected Button Element');
```

38. Modifying Text and HTML

Once we have selected an element from the DOM and stored it in a variable, we can change its contents. JavaScript provides properties on the element object to do exactly this.

innerText vs innerHTML

innerText : This property allows you to read or replace the raw text inside an element. It is safe and ignores HTML tags.

innerHTML : This property allows you to read or inject actual HTML tags into the element. While powerful, it can be dangerous if used with user-submitted data (Cross-Site Scripting attacks), so use it carefully!

Live Output:

Terminal / Console

```
> Old Title Text changed to: 'JavaScript is Awesome!'  
> HTML injected into the container.
```

Code Implementation:

```
// 1. Select the elements  
let title = document.getElementById('main-title');  
let box = document.querySelector('.content-box');  
  
// 2. Change raw text  
title.innerText = 'JavaScript is Awesome!';  
console.log("Title text changed");  
  
// 3. Inject HTML markup  
box.innerHTML = '<strong>Warning:</strong> System Error';  
console.log("HTML injected");
```

39. Modifying CSS Styles via JS

JavaScript can also manipulate the visual design of the page dynamically. There are two primary ways to do this:

1. Inline Styles (`element.style`)

You can directly apply CSS properties. Note: CSS properties with hyphens (like `background-color`) are converted to camelCase (`backgroundColor`) in JavaScript because hyphens signify subtraction in JS.

2. Class List (`element.classList`)

The better and cleaner approach is to have pre-defined CSS classes in your stylesheet, and use JavaScript to simply `add()` , `remove()` , or `toggle()` those classes on an element.

Live Output:

Terminal / Console

```
> Styles updated successfully.  
> Box is now Blue and has 'active' class.
```

Code Implementation:

```
let box = document.querySelector('.box');  
  
// Modifying Inline Styles  
// 'background-color' becomes 'backgroundColor'  
box.style.backgroundColor = 'blue';  
box.style.fontSize = '24px';  
box.style.display = 'block';  
  
// Using the ClassList API (Best Practice)  
// Assuming '.hidden' and '.active' exist in CSS  
box.classList.remove('hidden');  
box.classList.add('active');  
  
console.log('Styles updated successfully.');
```

40. Introduction to Events

A static webpage is boring. Interactivity requires the page to respond to the user. An **Event** is any action that happens in the browser—a mouse click, a key press, scrolling, or submitting a form.

Event Listeners

To make our JavaScript react to these events, we use a method called `addEventListener()`. We attach it to a DOM element, tell it what specific event to listen for (like 'click'), and give it a Function to execute whenever that event happens.

This is the core foundation of all interactive web applications!

Live Output:

Terminal / Console

```
> System standing by ...  
> (When user clicks the button)  
> Button was clicked! Executing code ...
```

Code Implementation:

```
// 1. Select the button element  
let submitBtn = document.querySelector('.submit-btn');  
  
// 2. Create the function that will run when clicked  
function handleClick() {  
    console.log('Button was clicked! Executing code ... ');  
}  
  
// 3. Attach the Event Listener to the button  
// Format: element.addEventListener('eventName',  
functionToRun);  
submitBtn.addEventListener('click', handleClick);  
  
console.log('System standing by ... ');
```

41. The Event Object

When an event occurs (like a mouse click or a key press), the browser automatically gathers detailed information about that event—such as the exact X and Y coordinates of the mouse, or which specific key was pressed. It packages all this data into an **Event Object**.

If you put a parameter (usually named `e` or `event`) inside your event listener function, JavaScript will automatically pass this Event Object into your function when the event triggers.

This is incredibly useful! For example, in a game, you need to know exactly *which* key the user pressed to move the character.

Live Output:

Terminal / Console

```
> (When user presses 'Enter')  
> Key pressed: Enter
```

Code Implementation:

```
// 1. Listen for any key press on the entire webpage (the  
window)  
window.addEventListener('keydown', function(event) {  
  
    // The 'event' object contains a property called 'key'  
    // which tells us exactly which button was pushed.  
    let pressedKey = event.key;  
  
    console.log('Key pressed:', pressedKey);  
  
    // Using an if statement based on the event data  
    if (pressedKey === 'Enter') {  
        console.log('Submitting form... ');  
    }  
});
```

42. Asynchronous JavaScript Introduction

JavaScript is a **Single-Threaded** language, meaning it can only execute one line of code at a time on a single main thread. If a piece of code takes 5 seconds to run (like downloading a large image), the entire webpage would freeze for 5 seconds!

The Solution: Asynchronous Programming

To prevent the browser from freezing, JavaScript offloads heavy tasks (like timers or fetching data from a server) to the browser's Web APIs. The browser handles the heavy lifting in the background, and when it finishes, it alerts JavaScript to execute a callback function.

This allows JavaScript to continue running other code smoothly while waiting for background tasks to finish.

Live Output:

Terminal / Console

```
> 1. Starting task ...  
> 3. Other code runs immediately!  
> 2. Data downloaded! (after 2 seconds)
```

Code Implementation:

```
console.log('1. Starting task ... ');  
  
// setTimeout is an Asynchronous Web API.  
// It tells the browser to wait 2 seconds in the background.  
setTimeout(function() {  
    // This callback function runs AFTER the 2 seconds.  
    console.log('2. Data downloaded!');  
}, 2000);  
  
// JavaScript DOES NOT stop and wait for setTimeout.  
// It immediately moves on to the next line of code.  
console.log('3. Other code runs immediately!');
```

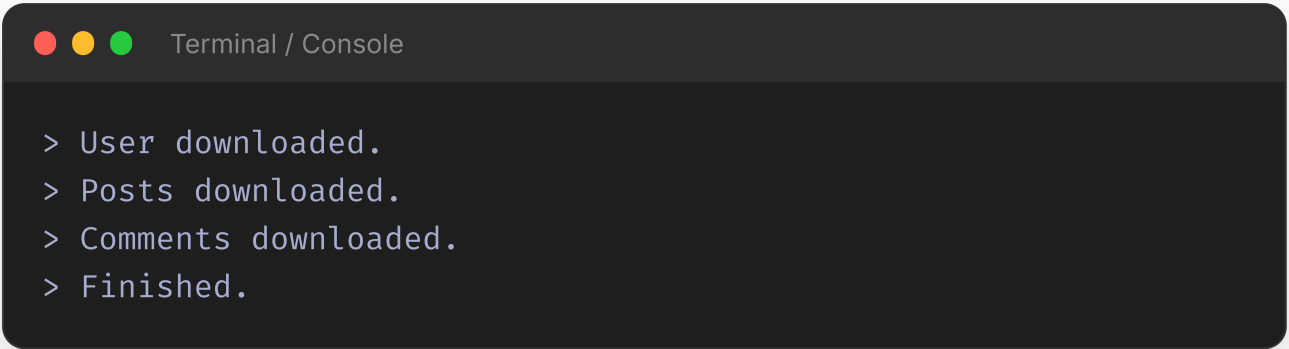
43. The Problem: Callback Hell

When dealing with Asynchronous code, we use "Callback" functions to tell the program what to do once the data is ready. But what happens if you need to fetch Data A, then use Data A to fetch Data B, and then use Data B to fetch Data C?

You end up nesting callback functions inside callback functions inside callback functions. This creates a massive, horizontal, triangle-shaped block of code known in the industry as **Callback Hell** or the *Pyramid of Doom*.

It makes the code incredibly hard to read, scale, and debug. To solve this, JavaScript introduced **Promises**.

Live Output:

A terminal window with a dark background and a title bar containing three colored circles (red, yellow, green) and the text "Terminal / Console". The terminal displays four lines of output, each preceded by a greater-than sign (>).

```
Terminal / Console

> User downloaded.
> Posts downloaded.
> Comments downloaded.
> Finished.
```

Code Implementation:

```
// A simulated example of Callback Hell
// Notice how the code shifts further to the right.

downloadUser(function(user) {
  console.log('User downloaded.');
```

```
  downloadPosts(user.id, function(posts) {
    console.log('Posts downloaded.');
```

```
    downloadComments(posts[0].id, function(comments) {
      console.log('Comments downloaded.');
```

```
      console.log('Finished.');
```

```
    });
  });
});
```

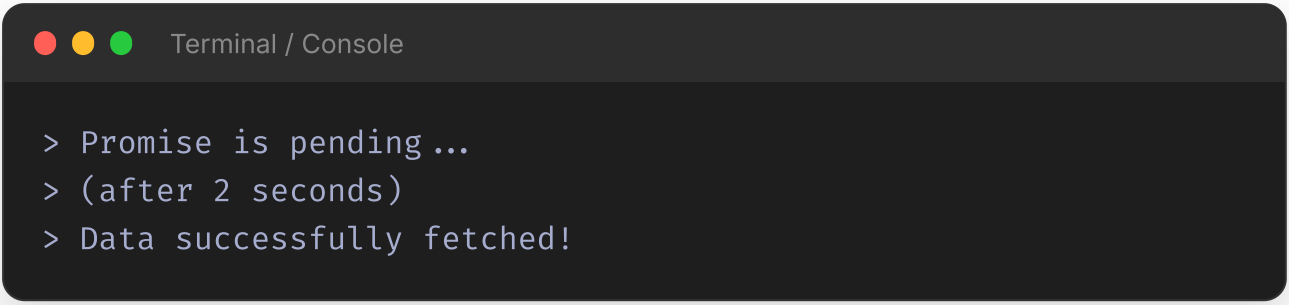
44. Introduction to Promises

A **Promise** is an object representing the eventual completion (or failure) of an asynchronous operation. Think of it like a restaurant buzzer: you place your order, and they hand you a buzzer. The buzzer is a "promise" that you will eventually get your food.

The 3 States of a Promise:

1. **Pending:** The initial state. The background task is still running. (You are waiting for the food).
2. **Fulfilled (Resolved):** The task completed successfully! (The buzzer rings, your food is ready).
3. **Rejected:** The task failed due to an error. (The kitchen ran out of ingredients).

Live Output:

A terminal window with a dark background and a title bar that says "Terminal / Console". It contains three lines of text: "> Promise is pending ...", "> (after 2 seconds)", and "> Data successfully fetched!".

```
> Promise is pending ...
> (after 2 seconds)
> Data successfully fetched!
```

Code Implementation:

```
// Creating a custom Promise
let myDataPromise = new Promise(function(resolve, reject) {

  // Simulating a network request that takes 2 seconds
  setTimeout(function() {
    let success = true;

    if (success) {
      resolve('Data successfully fetched!'); // Fulfills the
promise
    } else {
      reject('Network Error!'); // Rejects the promise
    }
  }, 2000);

});
console.log('Promise is pending ...');
```

45. Consuming Promises (.then & .catch)

Once you have a Promise (like the one returned by an API call), you need to handle what happens when it resolves or rejects. We "consume" promises using two chained methods.

.then()

This block of code executes ONLY if the Promise resolves successfully. The data passed into the `resolve()` function is sent here.

.catch()

This block of code executes ONLY if the Promise is rejected (e.g., the Wi-Fi dropped). This allows you to handle errors gracefully without crashing the app.

Live Output:

Terminal / Console

```
> Waiting for data ...  
> Success! Data successfully fetched!
```

Code Implementation:

```
console.log('Waiting for data ... ');  
  
// We use the Promise we created in the previous topic.  
myDataPromise  
  .then(function(resultData) {  
    // This runs if resolve() was called  
    console.log('Success!', resultData);  
  })  
  .catch(function(errorMessage) {  
    // This runs if reject() was called  
    console.log('Failed:', errorMessage);  
  });
```

46. Async and Await (Modern JS)

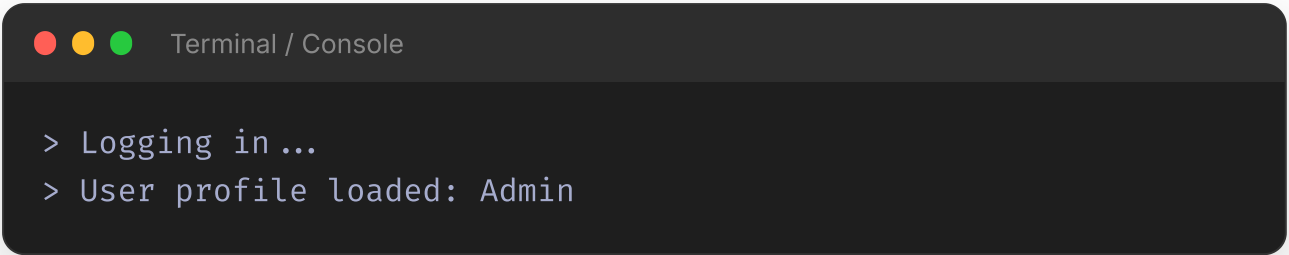
While `.then()` and `.catch()` solved Callback Hell, chaining them could still look messy. In 2017, JavaScript introduced **Async / Await**, which is syntactic sugar over Promises.

It allows us to write asynchronous code that *looks and behaves* like synchronous, line-by-line code!

How it works:

- Put the `async` keyword in front of a function to declare it as asynchronous.
- Inside that function, use the `await` keyword in front of any Promise.
- The function will literally **pause** execution at the `await` line until the Promise resolves, and then store the result in a variable.

Live Output:

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the text "Terminal / Console". The terminal shows two lines of output: "> Logging in ..." and "> User profile loaded: Admin".

```
Terminal / Console

> Logging in ...
> User profile loaded: Admin
```

Code Implementation:

```
// Simulating a promise that returns user data
function fetchProfile() {
  return new Promise(res => setTimeout(() => res('Admin'),
1000));
}

// The modern Async/Await syntax
async function handleLogin() {
  console.log('Logging in ... ');

  // The code pauses here until fetchProfile() finishes!
  let profileData = await fetchProfile();

  // This line only runs AFTER the promise is resolved
  console.log('User profile loaded:', profileData);
}

handleLogin();
```

47. Handling Errors with try/catch

When using Async/Await, we no longer use `.catch()`. Instead, we use standard **try/catch** blocks, which is how error handling is done in traditional languages like Java or Python.

You put the risky code (like fetching data from a server) inside the `try` block. If any Promise is rejected, JavaScript immediately stops the `try` block and jumps straight into the `catch` block, passing the error along.

This makes handling errors extremely clean and predictable.

Live Output:

Terminal / Console

```
> Starting data fetch...  
> An error occurred: Server is down!  
> Execution continues normally.
```

Code Implementation:

```
async function getDashboardData() {  
  try {  
    console.log('Starting data fetch ... ');  
  
    // Simulating a Promise that rejects (fails)  
    let response = await Promise.reject('Server is down!');  
  
    // This line will NEVER run because the promise rejected  
    console.log('Data loaded:', response);  
  
  } catch (error) {  
    // Execution immediately jumps here  
    console.log('An error occurred:', error);  
  }  
}  
  
getDashboardData();  
console.log('Execution continues normally.');
```

48. The Fetch API

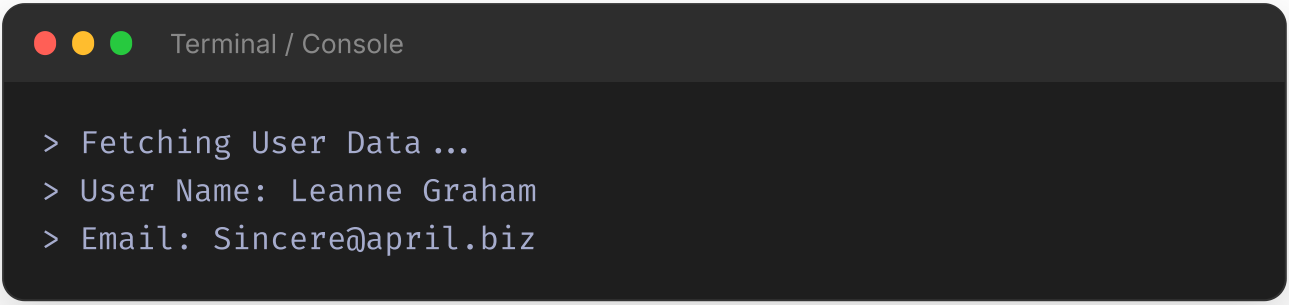
The **Fetch API** is a modern, built-in browser tool used to make HTTP network requests (like pulling data from a database or a third-party server). It completely replaces the older, clunky `XMLHttpRequest`.

`fetch()` requires a URL and automatically returns a Promise. When the server responds, the Promise resolves into a Response object.

The JSON Step

The raw response is not immediately usable in JavaScript. You must call `response.json()` to convert the incoming data packet into a native JavaScript Object. Note that `.json()` *also* returns a Promise, so you must `await` it twice!

Live Output:

A terminal window with a dark background and a title bar containing three colored circles (red, yellow, green) and the text "Terminal / Console". The terminal displays three lines of output: "> Fetching User Data ...", "> User Name: Leanne Graham", and "> Email: Sincere@april.biz".

```
Terminal / Console

> Fetching User Data ...
> User Name: Leanne Graham
> Email: Sincere@april.biz
```

Code Implementation:

```
async function getUser() {
  try {
    console.log('Fetching User Data ... ');

    // 1. Await the network request to the server
    let response = await
    fetch('https://jsonplaceholder.typicode.com/users/1');

    // 2. Await the conversion of the data into a JS Object
    let data = await response.json();

    // 3. Use the data!
    console.log('User Name:', data.name);
    console.log('Email:', data.email);

  } catch (error) {
    console.log('Failed to fetch:', error);
  }
}

// Uncomment to run in a real environment
// getUser();
```

49. Introduction to Object-Oriented JS

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects", which can contain data and code to manipulate that data.

Instead of writing 100 loose variables and functions, OOP allows you to bundle related variables and functions into a single logical entity. For example, a `BankAccount` object would hold the ``balance`` (data) and the ``deposit()`` function (behavior).

Classes (The Blueprint)

A **Class** is a blueprint or template for creating objects. You define the structure in the Class, and then you "instantiate" (create) multiple individual objects based on that blueprint.

Live Output:

```
Terminal / Console

> New account created for Alice
> Balance: 0
> New account created for Bob
> Balance: 0
```

Code Implementation:

```
// Defining the Blueprint (Class)
class BankAccount {
  // The constructor runs automatically when a new object is
  made
  constructor(ownerName) {
    this.owner = ownerName;
    this.balance = 0;
    console.log('New account created for ' + this.owner);
  }
}

// Creating new Objects (Instances) based on the Class
let account1 = new BankAccount('Alice');
console.log('Balance:', account1.balance);

let account2 = new BankAccount('Bob');
console.log('Balance:', account2.balance);
```

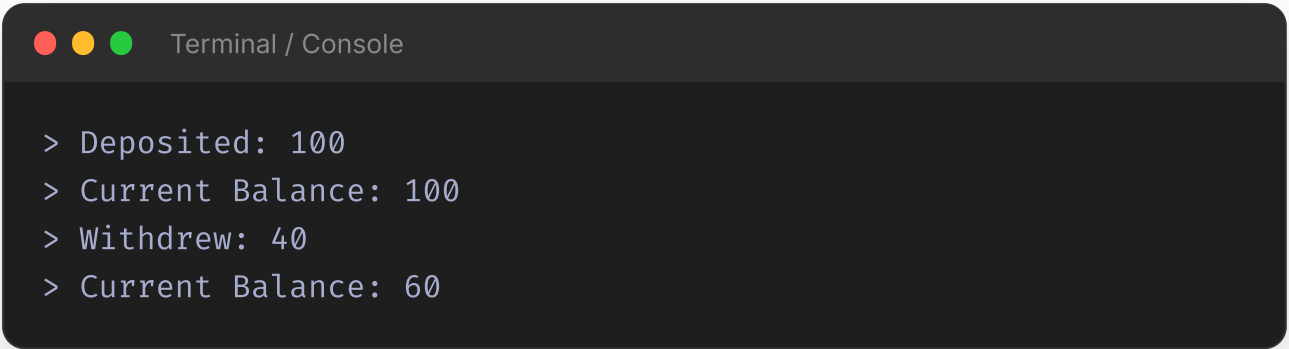
50. OOP Methods and Encapsulation

Once we have a Class, we want to add behaviors to it. We do this by defining **Methods** inside the Class block.

In a properly designed OOP system, outside code shouldn't manually change the internal data (like `account.balance = 5000`). Instead, the outside code should call methods (like `account.deposit(5000)`), and the object updates its own internal data.

This concept of hiding internal state and requiring interaction through defined methods is called **Encapsulation**. It prevents bugs and makes code highly predictable.

Live Output:



```
Terminal / Console

> Deposited: 100
> Current Balance: 100
> Withdrew: 40
> Current Balance: 60
```

Code Implementation:

```
class BankAccount {
  constructor(ownerName) {
    this.owner = ownerName;
    this.balance = 0;
  }

  // Method to handle deposits
  deposit(amount) {
    this.balance += amount;
    console.log('Deposited:', amount);
  }

  // Method to handle withdrawals
  withdraw(amount) {
    this.balance -= amount;
    console.log('Withdrew:', amount);
  }
}

let myAccount = new BankAccount('Nitesh');

// Using methods instead of changing this.balance directly
myAccount.deposit(100);
console.log('Current Balance:', myAccount.balance);
```

Want More Premium Notes?

Browse our premium collection of notes and project files tailored for students of all fields. Available exclusively at **programenote.dev!**

Programming Notes
(Btech/Bsc)

B-Tech Notes

Bsc Notes

BBA Notes

MBA Notes

D Pharma Notes

B Pharma Notes

LLB Notes

Science Notes (10th to 12th)

Commerce Notes

Arts Notes (12th to
Graduation)

Visit: www.programenote.dev